

Rapidly Build and Deploy a Full-Stack App with Cursor

Advanced AI-assisted SaaS application development

Lucas Soares
September 2026
Cursor 3

DEMO STACK

Next.js 15+ App Router

TypeScript

Tailwind + shadcn/ui

Supabase + Drizzle

Clerk

Stripe

Anthropic API · claude-sonnet-5

Vitest + Playwright

Vercel

1 CURSOR & CONTEXT MANAGEMENT

repo: sections 01–04

Four ways to work with the AI

Mode	Shortcut	Use it for
Chat	Cmd+L	Explore, explain, quick Q&A; multi-tab, context checkpoints
Agent	Cmd+I	Multi-file edits, autonomous execution, plan + build
Inline Edit	Cmd+K	In-place change from natural language, stay in flow
Tab	Tab	Multi-line, context-aware predictions

What's new in Cursor 3

- **Composer 2.5** — Standard + Fast tiers, an iteration over Composer 2
- **Agents Window** — parallel agents across local, worktree, cloud & SSH
- **Bugbot** — automated PR review on Composer 2.5, ~90-second reviews
- **Projects** (beta, Sept 2026) — a coordinator agent delegates to parallel subagents, each on its own cloud VM; closing your laptop doesn't stop it

1 EXPLORE

Share context, identify the files involved, discuss options. No code yet.

2 PLAN

Step-by-step plan as a Markdown checklist; break the work into small tasks.

3 BUILD

Execute iteratively, review each step, commit regularly.

@ symbols — your context navigator

@file	@src/components/Header.tsx	one specific file
@folder	@src/utils/	a whole directory
@codebase	@codebase "authentication"	semantic search
@docs	@docs "React hooks"	query documentation
@git	@git "last commit"	git history
@branch	@branch feature/new-auth	another branch

Context management

- Context windows are fixed (128k–1M tokens), and comprehension ≠ token count
- Poor context → hallucinations. More context is not better: curate it
- Use @-mentions for precision; start new chats often
- Keep noise out of the index with `.cursorignore`
- **Prompt the diff:** current state → desired state, with success criteria. "Make this better" has none

Project rules — `.cursor/rules/`

- **Always Apply** — every session, automatically
- **Apply Intelligently** — the AI decides when it's relevant (recommended)
- **Apply to Specific Files** — pattern match, e.g. `*.test.ts`
- **Apply Manually** — only when you @-mention it

Use for recurring mistakes, security patterns, team consistency. Keep each under 500 lines. Better context beats more rules. `AGENTS.md` (Cursor) and `CLAUDE.md` (Claude Code) carry project-level context.

Parallel agents on git worktrees

One clone, several working folders, each on its own branch, all sharing one `.git` history. Three agents → three folders → zero file collisions.

```
git worktree add ../auth feature/auth
# agent works in ../auth; review its diff
git worktree remove ../auth # after merging
```

vs. `git checkout`: one folder, one branch at a time; stash first, rebuild after each switch.

MCP — Model Context Protocol ("USB for AI")

An open protocol connecting the AI to tools and data. A server exposes **Tools** (functions the AI runs), **Prompts** (templated workflows) and **Resources** (structured data).

- **GitHub** — create PRs, review code, manage issues
- **Figma** — extract design tokens, generate components
- **GitMCP** — any repo as context: swap `github.com` → `gitmcp.io`
- **Databases** — query and inspect PostgreSQL / SQLite
- **Playwright** — the AI drives and sees your running app

MCP config — GitMCP for the Next.js docs

```
{
  "mcpServers": {
    "nextjs-docs": {
      "command": "npx",
      "args": ["-y", "mcp-remote",
        "https://gitmcp.io/vercel/next.js"]
    }
  }
}
```

Project config lives in `.cursor/mcp.json`. More examples: [sections/04-mcp-and-tooling/mcp-config-examples/](#)

2 BUILDING YOUR FIRST FULL APP

repo: sections 02, 05

1 DESCRIBE

Problem, users and features in plain English.

2 DEFINE

Tech stack, database, auth, deployment.

3 GENERATE

Let Cursor scaffold everything from the context file.

docs/context.md — the project's north star

```
# DeepWork AI - Focus Productivity App
## Problem
Developers struggle with multitasking and distractions.
## Users
Software engineers, designers, product managers
## Core Features
1. Task management with priority levels
2. Focus timer for single-task concentration
3. AI chat to add tasks via natural language
## Tech Stack
Next.js 15+ (App Router), TypeScript, Tailwind,
Supabase (database + auth), Vercel
## Database Schema
[Let Cursor generate based on features]

Then: Generate the project structure for @docs/
context.md
```

Two-stage workflow

1. **ChatGPT / Claude / Gemini** — **plan**: brainstorm features, structure ideas, write `context.md`, plan the architecture.
2. **Cursor** — **implement**: generate code, multi-file operations, debug errors, refactor.

Iterative development loop

1. Generate code with Agent
2. Run the app: `npm run dev`
3. Copy the error message into chat
4. Let Cursor fix it, then review the change
5. **Commit** the working state with git

Supabase backend in three moves

1. Create a project at [supabase.com](#); copy the API URL and anon key
2. Add `NEXT_PUBLIC_SUPABASE_URL` and `NEXT_PUBLIC_SUPABASE_ANON_KEY` to `.env.local`
3. Prompt: `@docs/context.md` + "install the Supabase client and create the schema for tasks with title, description, priority, deadline, completed, user_id"

Cursor generates the SQL migrations and RLS policies. Work component by component and review each before moving on.

Testing with AI

- **TDD**: write the test first (you define the behaviour), let the AI implement
- AI is good at edge cases you'd miss
- Keep the pyramid: many unit tests, fewer E2E
- Layers: component → integration → E2E (Vitest + Playwright)

```
@src/components/TaskCard.tsx
Create a complete test suite. Include rendering,
user interaction, edge case and accessibility
tests.
```

3 ADVANCED APP DEVELOPMENT & ARCHITECTURE

repo: section 07

Atomic design

Atoms Button, Input → **Molecules** SearchBar → **Organisms** Header → **Templates** layouts → **Pages** full views

State management

- Local — `useState`
- Global client — Zustand
- Server — React Query
- URL — `searchParams`

Hooks & performance

- Custom hooks: single responsibility, named exports, typed
- `React.memo`, `useMemo`, `useCallback`, code splitting

Architecture diagrams with Mermaid

Create a Mermaid diagram showing the data flow in our quiz app:
user selects topic → API generates questions → questions shown one at a time → answers stored in Supabase → results calculated and shown

`flowchart` `logic` · `sequenceDiagram` interactions · `classDiagram` data structures. Start small, layer upward, show multiple perspectives.

Figma MCP → React

Design tokens, component specs and typography in; pixel-accurate, responsive Tailwind components out.

@figma file:quiz-app-designs
Generate the QuestionCard component matching the Figma design. Extract all design tokens (colors, spacing, shadows) and create a matching React component with Tailwind.

4 REAL-WORLD FULL-STACK APPLICATION

repo: sections 05, 08

Prompt — a complete REST API

Create a complete REST API for our quiz app:
- POST /api/quiz/generate create new quiz
- GET /api/quiz/[id] get quiz by ID
- POST /api/quiz/[id]/answer submit answer
- GET /api/quiz/[id]/results final results
- GET /api/quiz/history user's past quizzes
Include: authentication, validation (Zod), error handling, rate limiting, API documentation comments

Database optimization

Indexes speed up queries · **JOINS** cut round trips · **Pagination** limits results · **Caching** via a Redis layer · **Connection pooling** via Supavisor

Optimistic updates — the app feels instant

1. Update the UI immediately
2. Send the request to the server
3. Error → roll the UI back; success → keep it

Zustand for quiz state · Supabase subscriptions for real-time sync

Environment variables (`.env.local`) — never commit this file

Service	Variables	Where to find them
Supabase	<code>DATABASE_URL</code> · <code>NEXT_PUBLIC_SUPABASE_URL</code> · <code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	Settings → Database / API
Clerk	<code>NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY</code> · <code>CLERK_SECRET_KEY</code>	Clerk Dashboard → API Keys
Stripe	<code>STRIPE_SECRET_KEY</code> · <code>NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY</code> · <code>STRIPE_WEBHOOK_SECRET</code>	Developers → API Keys / Webhooks
AI model	<code>ANTHROPIC_API_KEY</code> , or <code>OPENAI_API_KEY</code> / <code>DEEPSEEK_API_KEY</code>	Provider console → API Keys

5 DEPLOYMENT, TESTING & PRODUCTION

repo: sections 06, 09

Ship with Vercel

Zero config with automatic optimizations · **Edge network** global CDN · **Instant rollback** one-click revert.

CI/CD with GitHub Actions — ask Cursor to generate it:

- **Every PR:** type check → ESLint → unit + integration tests → build → Vercel preview deploy
- **Merge to main:** all of the above + E2E tests + production deploy

PR feedback, fast

1. **Screenshot** the PR comments
2. Drag into Cursor and **generate a plan**
3. **Execute** the fixes one by one

Even faster: run `/review` (`/review-bugbot` , `/review-security`) before you push. Bugbot integrates with GitHub and GitLab PRs.

Production readiness checklist

Security	Performance	Code quality	Monitoring & recovery
Env vars secured SSL + CORS configured Rate limiting enabled Dependency & secret scanning	CDN + edge caching Image optimization Code splitting Database indexes	Strict TypeScript ESLint + a11y checks 80% test coverage	Error tracking (Sentry) Uptime + analytics Automated backups Rollback strategy

Free tiers (what the demo runs on)

Vercel Hobby	100 GB fast data transfer/mo, 1M edge requests/mo
Supabase Free	500 MB database, 1 GB file storage, 50K monthly active users
Clerk Free	50,000 monthly retained users (MRU); a signup counts only once it returns 24h+ later
Stripe	No monthly fee; 2.9% + 30¢ per transaction

When you outgrow them

Vercel Pro	\$20/mo
Supabase Pro	\$25/mo
Clerk Pro	\$25/mo (\$20/mo annual), only above 50K MRU
Cheaper model tier	<code>gpt-5.6-luna</code> ~\$0.20 / 1M input tokens where quality allows

Troubleshooting — quick fixes

AI output is poor	Cut context, sharpen the prompt, check you're in the right mode
Context window full	New chat tab; summarize progress and hand it off
<code>useState</code> / event handler error	Add <code>"use client"</code> at the top of the file
Server Action not running	Check the <code>"use server"</code> directive
Build fails, dev works	Missing env vars, or server-only code in a static page

Keep these open

- Course repo — `github.com/EnkrateiaLucca/cursor-course`
- Cursor docs — `docs.cursor.com`
- McKay's App Template — `github.com/mckaywrigley/mckays-app-template`
- MCP servers — `github.com/modelcontextprotocol/servers`
- GitMCP — `gitmcp.io`

REMEMBER

1. Context is king — strategic @-mentions, clean project structure

2. Explore → Plan → Build — don't skip the first two

3. Rules encode your standards — add them only when mistakes repeat

4. MCP extends Cursor's reach — GitHub, Figma, docs, databases

5. Git discipline is non-negotiable — commit early, commit often

6. Try 5 times — rephrase and retry before giving up on a prompt

7. Production-ready means tests, CI/CD, monitoring and security

8. Cursor augments your judgment — it doesn't replace it